CPSC 321: Homework 5 (Dynamic SQL) *Due: Tuesday, Nov. 3, 2026*

Instructions. This homework asks you to implement a text-based menu program over your HW-3 and HW-4 "CIA Factbook" tables. The program may be written in either Python or Java using the provided starter code. Submit your assignment files to Canvas by midnight at the end of the due date. Submit the following source files for your chosen language along with a PDF write-up:

- If using Python: `factbook_ops.py` and `hw5.py`
- If using Java: `FactbookOps.java` and `HW5.java`
- `hw5.sql`: your script to recreate and populate the underlying tables
- `hw5.pdf`: your test inputs and output as described in Part 2 below

Homework Details

Starter Code. The starter code is provided in the `hw5-starter.zip` package on Canvas. You must implement six of the menu actions (one is already implemented for you) and their corresponding database operations described in Part 1. You may not modify the supplied filenames, function/method signatures, or menu choices.

Database Setup. Create a `hw5.sql` file using your factbook schema and data from HW-4. Include the commands needed to recreate and populate the `country`, `province`, `city`, and `border` tables, keeping all keys, foreign keys, and required attribute constraints. For HW-5, you must also include a province name that occurs in two countries and enough cities that at least one filtered search returns more than five rows. You must also include enough data to test population boundaries and sorting. As in HW-3 and HW-4, your data does not have to be based on actual locations or use real area, GDP, inflation, or population values. Be sure to re-run your `hw5.sql` script to reset the database before recording your tests in Part 2. Use the comment header format from HW-4, updated for HW-5.

Parameter Binding. Every *user-supplied input value* used in SQL must be passed as a parameter. Use `%s` placeholders with psycopg, or `?` placeholders and `PreparedStatement` setters with JDBC. ***Violating this rule will result in a score of 0 for HW-5.*** Note that you may need to use string concatenation for other parts of an SQL statement (e.g., to add where clause filters for the search cities operation), however, this must not include user input values directly, which must be handled by query parameter placeholder symbols instead.

Allowed SQL. Use the SQL covered in class, including `SELECT`, `INSERT`, `UPDATE`, and `DELETE`. Do all filtering and sorting in SQL. Do not use SQL constructs that have not been discussed yet in class. You may assume valid user input, including for integers and floating-point values.

Python Setup. If you choose to implement the assignment in Python, you must have Python 3.10 or later installed along with `psycopg` 3. The lecture notes provide installation examples. In the starter code's `python` directory, copy `db_config.py.example` to `db_config.py` and fill in your database connection settings. Run the program from that directory using `python hw5.py`.

Java Setup. If you choose to implement the assignment in Java, you must have JDK 17 or later installed. The starter code provides the PostgreSQL JDBC driver. In the starter code's

java directory, copy db_config.properties.example to db_config.properties and fill in your database connection settings. Compile and run your code from that directory:

```
javac HW5.java
java -cp .:postgresql-42.7.14.jar HW5
```

On Windows, you may need to use the following java command to run your program once the code is compiled:

```
java -cp ".;postgresql-42.7.14.jar" HW5
```

Java Records and Optional Values. In FactbookOps.java, a Country record is defined to hold a country's code, name, GDP, and inflation. In Java, record types automatically provide a constructor and accessor methods:

- To construct a new record use new Country(code, name, gdp, inflation).
- To read the attributes of a Country object c use c.countryCode(), c.countryName(), c.gdp(), and c.inflation().
- To refer to the Country type itself (e.g., in HW5.java), use FactbookOps.Country.

Additionally, the HW-5 starter code uses Java's Optional type:

- An Optional<Country> holds one Country record or is empty (i.e., has no value).
- Use Optional.of(c) to create an optional containing the record c, and Optional.empty() to create an empty optional.
- For an optional object o, use o.isPresent() to check whether it contains a value. Use o.get() to obtain its value (when it has one).

JDBC Methods. Given a PreparedStatement object ps, use ps.executeQuery() for a query and ps.executeUpdate() for an insert, update, or delete. Note that parameter positions start at 1 and values can be bound to statements using the appropriate setter, such as ps.setString(), ps.setInt(), or ps.setDouble(). The ps.setObject() method can also be used to bind values stored in an Object variable.

PART 1. Implementation

Each menu feature below must be implemented as a menu *action* and a database *operation*. Each action function goes in hw5.py or HW5.java and each operation goes in factbook_ops.py or FactbookOps.java. Actions read user input, open a connection, call the underlying operation, and display the results. Keep all terminal input/output in actions and all SQL statements and their execution in operations.

For operations that return a Boolean value, *true* means the requested write succeeded in the current transaction. The caller must still commit the transaction, as described below. A return value of *false* means no change was made because, e.g., the country already exists or the border does not exist. A false return value does not represent the case where an API error was raised.

Connections and Cursors. Each action is responsible for closing its connection after calling the underlying operation and processing any results. Operations must not commit, roll back, or close the connection passed to them. Operations also must not hide database exceptions. If an operation catches an exception to close a cursor or statement, it must raise the exception again in Python or rethrow it in Java (to pass it along to the caller).

In Python, the list countries and find cities operations receive a cursor created by the action. These operations execute their queries but do not fetch rows or close the cursor. The action fetches the results and closes the cursor. All other operations receive a connection and use a local cursor within a `with` block.

In Java, each operation receives a connection. For operations that return Booleans or `Optional<Country>` values, the statements and any result sets created must be closed before returning. Alternatively, the list-countries and find-cities operations must return an open `ResultSet` for the action to read and close. See the supplied `listCountries` example, which shows how to close the statement automatically when the action closes the result set.

Commit and Rollback. In Python, a connection's `with` block commits the transaction when it exits normally and rolls back automatically if an exception occurs. In the Java starter code, the `connect()` helper disables autocommit. This means that each action that writes to the database must call `cn.commit()` after its operation completes. Java's `try`-with-resources closes the connection but does not automatically commit. For the Java province-with-city action, you must catch an `SQLException` inside the connection block, roll back, and rethrow the exception:

```
} catch (SQLException e) {
    cn.rollback();
    throw e;
}
```

The connection must still be open when `rollback()` is called. This undoes both inserts if the operation cannot complete. In either language, report a successful write only after the transaction has committed.

Menu Features. In the examples below, the menu options are not shown, and the output is based on example data. Your database does not need to include these particular countries, etc.

1. **Show a country.** For this menu item, the show country *action* must read a country code and display the matching country in the same format as in the list-countries action of the starter code. The corresponding get-country *operation* must look up the country with the given code. In Python, the operation returns a tuple containing the country's code, name, GDP, and inflation, or `None` if no row matches. In Java, the operation returns an `Optional<Country>` containing a `Country` record, or an empty optional value. As an example, suppose your database contains the country Canada with code `CA` having a GDP per capita of 45000, and an inflation of 2.5. Selecting menu option 2 would produce:

```
Enter your choice (1-8): 2

Country code: CA

Canada (CA), per capita gdp $45000, inflation rate 2.5%
```

If the code does not occur in your database, then print a message that no matches were found:

```
Enter your choice (1-8): 2

Country code: XX

No matching country.
```

2. **Add a country.** For this menu item, the add-country *action* must read a country code, name, GDP, and inflation, then report whether the country was added or already exists. The corresponding add-country *operation* must first check whether the country code exists. If it does, it must return false and leave the row unchanged. Otherwise, it must insert the country and return true. For example, suppose no country with code `NZ` exists:

```
Enter your choice (1-8): 3

Country code: NZ
Country name: New Zealand
Country gdp: 42000
Country inflation: 2.0

Country added.
```

A second try with the same code must leave the original country's name, GDP, and inflation unchanged:

```
Enter your choice (1-8): 3

Country code: NZ
Country name: New Zealand Revised
Country gdp: 50000
Country inflation: 3.0

Country already exists.
```

3. **Update a country.** For this menu item, the update-country *action* must read a country code, GDP, and inflation, then report whether the country was updated or does not exist. The corresponding update-country *operation* must set the GDP and inflation of the country with the given code. Use the resulting row count to return true if the row exists and false otherwise (see the lecture notes for examples). An update to the values already stored still returns true. For example, if country `CA` exists:

```
Enter your choice (1-8): 4

Country code: CA
Country gdp: 46000
Country inflation: 2.0

Country updated.
```

If no country with code `AB` exists, no row is updated:

```
Enter your choice (1-8): 4

Country code: AB
Country gdp: 46000
Country inflation: 2.0

Country does not exist.
```

4. **Remove a border.** For this menu item, the remove-border *action* must read two country codes, then report whether the border was removed or does not exist. The corresponding remove-border *operation* must delete the border between the given countries, regardless of the order of their codes in the underlying table. Use the resulting row count to return true if a row was deleted and false otherwise. As in HW-3, store at most one row for each pair of bordering countries. For example, suppose the border is stored with `CA` as the first code and `US` as the second. Entering the codes in reverse order must still remove it:

```
Enter your choice (1-8): 5

Country 1 code: US
Country 2 code: CA

Border removed.
```

Repeating the action must report that the border no longer exists:

```
Enter your choice (1-8): 5

Country 1 code: US
Country 2 code: CA

No border exists between the countries.
```

5. **Add a province and city.** For this menu item, the add-province-with-city *action* must read a country code, province name, area, city name, and integer population, then report whether both were added or the province already exists. The corresponding add-province-with-city *operation* must check whether the province exists in the given country. If it does, the operation must return false without adding a city. Otherwise, it must insert the province, then its city, and return true. Both inserts use the given connection and belong to the same transaction. If

either insert raises a database exception, the operation must not return false. The action must roll back the transaction, undoing any rows inserted by the operation.
For example, suppose your database contains country `CA` but no province named Alberta in that country. The following action adds the province and its first city:

```
Enter your choice (1-8): 6

Country code: CA
Province name: Alberta
Province area: 255541
City name: Calgary
City population: 1678073

Province and city added.
```

If the province already exists, the action must not add another city or change the province's area:

```
Enter your choice (1-8): 6

Country code: CA
Province name: Alberta
Province area: 661848
City name: Edmonton
City population: 600000

Province already exists.
```

6. **Search cities.** For this menu item, the search-cities *action* must read four optional search criteria (i.e., search "facets"): country code, province name, minimum population, and maximum population. A blank input (e.g., if the user just presses enter) omits that filter. Pass omitted filters to the corresponding search cities *operation* as `None` in Python or `null` in Java. The search-cities operation must select the city name, province name, country code, and population for cities matching all supplied filters. Country codes and province names must match exactly, and population bounds are inclusive. If no filters are given, select all cities. Sort by population in descending order, then country code, province name, and city name in ascending order.
The search cities action must display each matching city's name, province, country code, and population, up to five cities at a time. After each batch, let the user continue or stop. If no cities match, display a message indicating this. It is okay to ask whether to continue if there are no more results.
For this search, assume that the result may be too large to reasonably collect in a list or array. In Python, the action must create a named cursor and pass it to the operation. The operation executes the query and returns `None`, leaving the action to fetch the rows. In Java, the operation must call `setFetchSize(5)` on its prepared statement before executing the query, then return the unread `ResultSet`. Execute the query once and keep the connection and cursor or result set open until all results have been displayed or the user chooses to stop.
For example, suppose the database contains the seven matching cities shown below. Leaving

the province blank allows cities from any province in `CA`. Entering `y` displays the next batch, and entering `n` returns to the menu:

```
Enter your choice (1-8): 7

Country code (enter to skip): CA
Province name (enter to skip):
Minimum population (enter to skip): 1000
Maximum population (enter to skip): 7000

Birch, Alberta (CA), population 7000
Cedar, Alberta (CA), population 6000
Elm, Alberta (CA), population 5000
Maple, Alberta (CA), population 4000
Oak, Alberta (CA), population 3000

See more rows (y/n): y

Pine, Alberta (CA), population 2000
Willow, Alberta (CA), population 1000

See more rows (y/n): n
```

If your database has no province named FooBar in country `CA`, the following search returns no rows. Both population filters are left blank:

```
Enter your choice (1-8): 7

Country code (enter to skip): CA
Province name (enter to skip): FooBar
Minimum population (enter to skip):
Maximum population (enter to skip):

No matching cities.
```

PART 2. Testing

Create a `hw5.pdf` write-up with your first and last name, the course (CPSC 321, Fall 2026), and the assignment (HW-5). For each menu item, include the inputs and output from your tests. Copy terminal text using a monospaced font (screenshots are not required).

Choose tests that demonstrate that each menu action works correctly. Include successful cases and unsuccessful cases, such as a missing country or an attempt to add an existing province. For actions that modify the database, verify the resulting changes. For the city search, test several combinations of filters, including omitted filters and a search with no matching cities. Also demonstrate displaying another batch of results and stopping before all results have been displayed. You do not need to test every combination of filters.